HyperSteer-Weight: Weight Steering for LLMs with Hypernetworks

Candidate Numbers: 60103 (25%), 71017 (25%), 71233 (25%), 74081 (25%)

Abstract

Steering is a promising approach to eliciting latent capabilities in Large Language Models (LLMs), inducing high-level behaviors such as domain-specific reasoning through lightweight interventions. However, most methods rely on steering vector discovery, limiting scalability and preventing generalization to unseen tasks. Recent work, such as HyperSteer, addresses this by employing hypernetworks for prompt-based activation steering. In this paper, we introduce HyperSteer-Weight, a steering method adapted to work in the parameter space by predicting coefficient vectors that reweight shared low-rank (LoRA-style) bases, producing prompt-conditioned weight updates for a frozen base model at deployment without task-specific gradient optimization. We evaluate HyperSteer-Weight on (i) the leading model-steering benchmark AxBench, where it outperforms simple baselines such as DiffMean and vanilla SAE variants, and (ii) AlpacaEval, showing generalization beyond AxBench to diverse instruction-following prompts. These results suggest a more deployment-efficient paradigm for LLM adaptation: train a single generator once and produce prompt-conditioned weight updates at deployment without per-concept tuning.

Keywords: large language models, hypernetworks, weight-space steering, LoRA

1. Introduction

1.1 Background: the problem of LLM adaptation

Large Language Models (LLMs) based on Transformer (Vaswani et al. (2017)) decoders have seen widespread adoption. They work as follows: upon the tokenization of the *input sequence* (the prompt), the tokens are mapped to *embeddings*, and processed by a stack of *layers* containing multi-head self-attention blocks and feed-forward networks, connected by residual connections. At each step, the model produces *logits* which parameterize the probability distribution for the next token via softmax. During *pre-training*, when LLMs are exposed to a massive corpus of text data, they acquire broad linguistic and world knowledge, and can exhibit *latent capabilities*, such as few-shot generalization and multi-step reasoning, whose expression in the output is often reliant on appropriate elicitation or adaptation (Brown et al. (2020)), motivating research into methods that can more reliably harness these capabilities for downstream use (Wang et al. (2023)).

Prompt engineering is a major paradigm, intending to elicit desirable high-level behaviors, such as Chain-of-Thought (CoT) reasoning, by means of optimizing the length and content of the input sequence (Wei et al. (2022b)). This approach offers an effective, but black-box and hard to scale, method for influencing LLM behavior (Cao et al. (2024)).

Fine-tuning addressed the scalability issue by updating the model's weights via gradient descent on a curated, often domain-specific, dataset, allowing for task-targeted LLM adap-

tation, with methods such as *reinforcement learning with human feedback* (RLHF) (Ouyang et al. (2022)) and *supervised instruction fine-tuning* (SIFT) (Wei et al. (2022a)), utilized to further align the outputs with user preferences. In practice, this requires voluminous domain-specific datasets while being computationally prohibitive (Zhang et al. (2024)), and often motivates switching between several discrete domain-specialized models for optimal performance (Wang et al. (2024)). High computational requirements motivated the introduction of *parameter-efficient fine-tuning* (PEFT) methods, such as *adapters* (Houlsby et al. (2019)), *prefix tuning* (Li and Liang (2021)), and *low-rank adaptation* (LoRA) (Hu et al. (2022)), targeting a compromise between performance and cost by updating only a small share of the model's parameters. While more efficient than full fine-tuning, these methods typically still require large curated datasets and a per-domain gradient-based optimization, underscoring the need for more flexible methods.

1.2 Related work: model steering

Steering represents a recent paradigm, aiming to induce domain-appropriate behaviors through interpretable, lightweight interventions, without task-specific training at deployment. Steering methods can be divided into *activation steering* and *weight steering*.

1.2.1 Activation Steering

Activation steering makes inference-time interventions in the model's internal representations (activation space) by adding *steering vectors* to the *residual stream*. Methods differ primarily in how these vectors are obtained.

Contrastive activation addition follows a *mean difference* approach constructing steering vectors as the average difference of residual stream activations between pairs representing positive and negative examples of a particular alignment behavior (Rimsky et al. (2024)). Unsupervised *dictionary learning* approaches, including sparse autoencoders trained on model activations, aim to discover collections of interpretable features which can be used for steering and concept detection (Huben et al. (2024), Joshi et al. (2025)).

The largest model steering benchmark, AxBench, shows that these representation-based methods generalize to thousands of concepts, but reveals large performance deficits relative to prompt engineering and fine-tuning baselines (Wu et al. (2025)). The above methods are also rigid in that they require pre-discovered steering vectors, or pre-learned/carefully selected concepts, highlighting the need for more flexible, prompt-conditioned interventions.

These can be obtained by using hypernetworks, auxiliary models used to generate parameters for the primary model (Ha et al. (2017)). HyperSteer applies this idea to scale activation steering, by having a hypermodel generate steering vectors conditioned on a *steering prompt* (natural-language description of the concept) and the activations of the base model, achieving strong generalization to unseen concepts and state-of-the-art performance on AxBench (Sun et al. (2025)).

1.2.2 Weight Steering

Weight steering intervenes in the parameter space, directly applying task-specific weight changes to the base model, without imposing additional per-token inference time overhead once applied. *Task vectors*, weight differences between a fine-tuned model and a pre-trained

variant, specify a direction in the parameter space, such that movement along this direction improves task performance. *Task arithmetic* shows that task vectors support addition, negation, and combination, work across open-source models, and succeed in tasks such as text generation and image classification (Ilharco et al. (2023)). Recently, *contrastive weight steering*, where task vectors are derived from the parameters of two fine-tuned models, one towards a positive behavior and one towards the negative, was applied to isolate behaviors such as sycophancy and misalignment, achieving stronger out-of-sample control than activation steering while preserving task performance (Fierro and Roger (2025)).

While parameter-space methods can generalize more robustly than activation steering, obtaining weight updates typically requires new fine-tuning runs per task, pointing to the need for a solution capable of generating them without task-specific training. A hypernetwork approach to generating LLM weights for specific downstream tasks was applied to a T5 encoder-decoder model, achieving strong performance in multi-task fine-tuning (Phang et al. (2023); Volk et al. (2023)). However, to the best of our knowledge, this hypernetwork approach has not yet been extended to concept steering in decoder-based LLMs.

Overall, the literature suggests a gap for a concept steering method that (i) provides prompt-conditioned domain-specific adaptation, (ii) does not require task-specific training at deployment, (iii) operates in the weight space, approximating the effects of PEFT.

1.2.3 HyperSteer-Weight

Inspired by HyperSteer and task vector arithmetic, we introduce *HyperSteer-Weight*, a hypernetwork weight steering method which maps a steering prompt to a LoRA-style, task-specific parameter update which can be applied to the base model. HyperSteer-Weight generates task vectors on demand: rather than maintaining separate fine-tuned models for each domain, we train a hypernetwork once and generate the task-appropriate weight edits at inference time. This yields a form of *continuous routing* in the parameter space, where the base model can be specialized to the task at hand without training at deployment.

1.2.4 Contributions

We make the following contributions:

- **Method.** We introduce HyperSteer-Weight, a method that generates LoRA weight updates from a steering prompt for a frozen base LLM with O(1) asymptotic time complexity at inference.

- **Empirical evaluation.** We evaluate HyperSteer-Weight on the established steering benchmark AxBench, scoring competitively, and we verify generalization to unseen domains on AlpacaEval.

2. Problem Formulation

We formulate the problem of LLM hypernetwork-based weight steering as a supervised learning problem (as seen in ST310) in the context of standard autoregressive generation.

Let B_θ be a pre-trained LLM with L layers and hidden dimension d, parameterized by frozen weights $\theta \in \mathbb{R}^p$. Let $\mathcal{V}$ denote the *vocabulary* (the set of tokens the model can produce

in one step) and let $\mathcal{V}^*$ be the set of all finite sequences of tokens from $\mathcal{V}$. Given an input sequence $x = (x_1, \ldots, x_T) \in \mathcal{V}^*$ of length T, the LLM defines the next-token probabilities $p_\theta(v \mid x_{1:t})$ for $v \in \mathcal{V}$ and $t \in \{1, \ldots, T\}$.

In fine-tuning, a task-specific update to θ is obtained by minimizing the empirical cross-entropy (negative log-likelihood) over a curated dataset $\mathcal{D}_{FT}$. In weight steering the weight adjustment is instead conditioned on a steering prompt $s \in \mathcal{V}^*$, a short natural-language description of the domain (e.g. 'number theory' or 'french').

In HyperSteer-Weight, for a steering prompt $s \in \mathcal{V}^*$ (the concept), this prompt-conditioned weight update is generated using the function $HyperSteer\text{-}Weight_\phi : \mathcal{V}^* \to \mathbb{R}^p$, parameterized by hypernetwork parameters ϕ. The steered model uses weights $\theta_{steer} = HyperSteer\text{-}Weight_\phi(s)$. Optionally, HyperSteer-Weight might also attend to the input prompt with cross attention, but this is not required.

Given a steering dataset $\mathcal{D}_{steer}$ containing triplets (x, s, y) where $x \in \mathcal{V}^*$ is the input sequence and $y = (y_1, \ldots, y_K)$ is a target (steered) continuation, we optimize HyperSteer-Weight to minimize the cross-entropy (CE) loss:

$$\mathcal{L}_{CE}(\theta_{steer}; x, y) := -\frac{1}{K} \sum_{t=1}^{K} \log \left[p_{\theta_{steer}}(y_t \mid x, y_{1:t-1}) \right].$$

The optimization task (this is a supervised training objective, as seen in ST310) is minimizing the expected CE loss across the steering dataset $\mathcal{D}_{steer}$:

$$\min_\phi \mathbb{E}_{(x,s,y)\sim\mathcal{D}_{steer}} \left[\mathcal{L}_{CE}\left(\theta_{steer}; x, y\right) \right].$$

3. Proposed Solution

We introduce **HyperSteer-Weight**, a weight steering method enabling on-the-fly model specialization. Unlike standard fine-tuning, which requires expensive gradient updates, our method generates task-specific parameters via a single forward pass of an auxiliary network.

3.1 Architecture and Parameterization

Hypersteer-Weight is decomposed into two components: a set of global learnable low-rank bases (shared across all tasks) and a lightweight hypernetwork that predicts coefficients to recombine these bases to steer towards a specific concept.

Steered Parameters. Let B_θ be a Transformer decoder with L layers and hidden dimension d. We target the Multi-Layer Perceptron (MLP) submodules, as these components are widely hypothesized to store factual knowledge (Meng et al. (2022), Dai et al. (2022)). We select a subset of layers $\mathcal{L}_{\text{steer}} \subseteq \{1, \ldots, L\}$. For each targeted layer ℓ, we intervene on the up-projection $W_{\text{up}}^{(\ell)} \in \mathbb{R}^{d_{\text{ff}} \times d}$ and down-projection $W_{\text{down}}^{(\ell)} \in \mathbb{R}^{d \times d_{\text{ff}}}$, where d_{ff} is the expansion dimension.

Global LoRA Bases. Predicting high-dimensional weight matrices is computationally intractable. Instead, for every steered layer ℓ, we initialize and learn static low-rank bases:

$$U_{\text{up}}^{(\ell)} \in \mathbb{R}^{d_{\text{ff}} \times r}, \quad V_{\text{up}}^{(\ell)} \in \mathbb{R}^{r \times d}, \qquad U_{\text{down}}^{(\ell)} \in \mathbb{R}^{d \times r}, \quad V_{\text{down}}^{(\ell)} \in \mathbb{R}^{r \times d_{\text{ff}}}.$$

These matrices are trained jointly with the hypernetwork but remain frozen at deployment. Intuitively, the columns and rows of these matrices form a 'dictionary' of possible weight modifications. The hypernetwork's role is to determine how much to activate each direction.

Hypernetwork. The hypernetwork H_ϕ is instantiated as a lightweight Transformer decoder, it processes a steering prompt $s \in \mathcal{V}^*$. H_ϕ generates (optionally also attending to the base LLM's hidden states through cross-attention) a context-aware coefficient vector:

$$c(s) \in \mathbb{R}^r$$

This allows the steering to be conditioned not just on the static instruction ('act like a biologist'), but dynamically on the specific input context. We only do this for one prompt per concept in inference to maintain O(1) time complexity.

Prompt-Conditioned Weight Updates. The final weight update is constructed by scaling the global bases by the generated coefficients. Formally, the update is:

$$\Delta W(s) := U \operatorname{diag}\big(c(s)\big) V.$$

The base model then executes its forward pass using the steered layer weights $W_{\text{steer}} = W + \Delta W$. We provide implementation details about weight merging in Appendix E.

Steering Intensity. To control the strength of the intervention at inference time (as done by HyperSteer), we introduce a scalar factor $f \geq 0$. We apply this by simply scaling the predicted coefficients: $c \leftarrow f \cdot c$. Setting $f = 0$ yields exactly the original unsteered model.

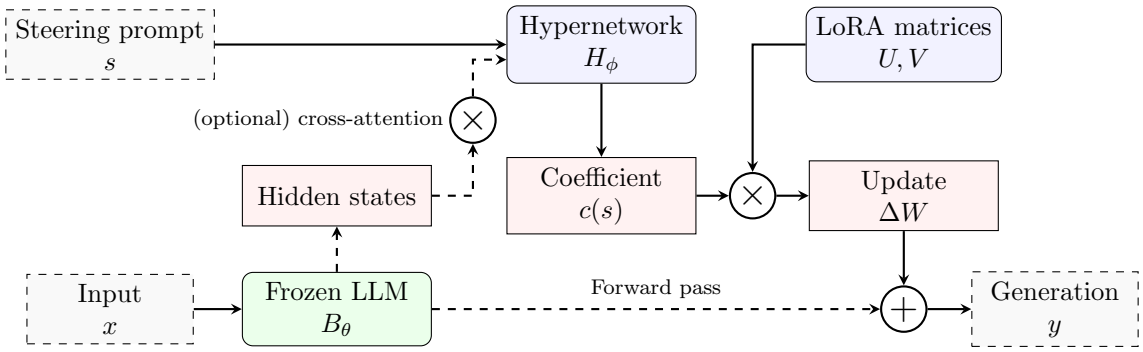


Figure 1: Architecture Overview: HyperSteer-Weight takes the steering prompt s to predict ΔW for the frozen LLM. Inputs and outputs are in grey, trained components are in blue, results of intermediate computations are in red, and the base LLM is in green.

4. Theoretical Formulation (Work in Progress)

Brief idea: provide game-theoretical justification and model for the policy gradient of the hypernetwork adapting meta-MAPG theorem of Kim et al. (2021). To achieve this, we need to model the steering game as a repated cooperative game with two actors: base model that is not aware of the other player and wants to maximise its reward (minimize loss), and a meta-aware hypersteer network whose reward depends on the reward (performance) of the base model, albeit the correspondence is not one-to-one.

To model this game, we need to introduce the tuple :

n-agent game is a tuple $\mathcal{I}$ $M_n = \langle \mathcal{I}, \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ where I - n agents (in our case, it is $\mathcal{I}_2$), $\mathcal{A} = \times_{i \in \mathcal{I}} \mathcal{A}_i$ - set of action spaces for each agent (assuming action space remains constant)

γ is the discount factor

$\mathcal{S}$ is the state space. It is game dependent and can be, for example, set of all past actions on a game. In general, it describes the state of the game at the current step (what happened before and is now). In our case, the state will describe the steering prompt, the input, the output, and current parameters of both models.

Reward function of the agent $r_t^i = \mathcal{R}^i(s_t, a_t)$ depends on the current state of the game and actions of all agents at a current step. At the end of an **episode** (what is it?) agent collects a trajectory - the path of all actions, states and rewards, and parameters of all policies

We need to:

- Formalize the reward function for our game
- Define what are parameters of the models (Mateusz, please read Sutton and Barto (2018), chapter 3 and 13. I can send you a pdf)
- Formalize the policies of our models, and apply the multi-mapg following Kim et al. (2021)
- Having arrived at a gradient, compare it with the current design of hypersteer that marco has, and to adapt hypersteer as needed
- In the meantime, Marco should read the experimental part of Kim et al. (2021) and come up with proposal of what specific experiments we want to run, similar to the experiments of Kim. Ideally, we would follow their style and answer specific questions (for example, how well does it generalize).
- Have a party once we are done - this is fucking intense but we can do it

4.1 Complexity Analysis and Overhead

A distinct advantage of HyperSteer-Weight is its asymptotic efficiency. We analyze computational complexity relative to standard autoregressive generation and activation steering.

Algorithm 1 Inference with HyperSteer-Weight

Input: Steering prompt s, Input x, Base B_θ, Hypernetwork H_ϕ, LoRA Bases $\{U, V\}$
$h_x \leftarrow B_\theta(x)$ {optionally extract hidden states for H_ϕ to attend to with cross-attn}
$c \leftarrow H_\phi(s)$
for each steered layer ℓ **do**
  $\Delta W^{(\ell)} \leftarrow U^{(\ell)} \cdot \mathrm{diag}(c) \cdot V^{(\ell)}$
  $W^{(\ell)}_{steer} \leftarrow W^{(\ell)} + \Delta W^{(\ell)}$
end for
return $B_{\theta_{steer}}(x)$

Inference Complexity. Let B_θ be the base LLM with hidden dimension d, depth L, sequence length T, and vocabulary size V. For standard autoregressive generation, the computational cost is dominated by position-wise dense matrix multiplications in the feed-forward networks (FFNs) and attention projections. While self-attention technically scales quadratically with sequence length (Keles et al., 2023), in the regime where d is large, the per-token compute is dominated by the $O(d^2)$ operations in the FFNs (Lin et al., 2022). Thus, the total floating-point operations (FLOPs) for generating a sequence of length T scale as $O(T \cdot L \cdot d^2)$ (Vaswani et al., 2017).

In HyperSteer-Weight, the inference process is decomposed into a *setup phase* and a *generation phase*.

Setup Phase (Amortized Cost): First, the hypernetwork H_ϕ processes the steering concept s (length $T_s \ll T$). With hypernetwork depth L_{hyp} and dimension d_{hyp}, this incurs a fixed cost of $O(T_s \cdot L_{\text{hyp}} \cdot d^2_{\text{hyp}})$. Second, the low-rank matrices U and V are multiplied to reconstruct the updates $\Delta\theta = UV$. For a targeted subset of layers $|\mathcal{L}_{\text{steer}}|$, this cost scales as $O(|\mathcal{L}_{\text{steer}}| \cdot d^2 \cdot r)$. Crucially, this operation is performed only *once* per concept, and the resulting weights are merged into the base model ($\theta_{steer} \leftarrow \theta + \Delta\theta$) prior to generation.

Generation Phase (Zero Marginal Cost): Once θ_{steer} is loaded, the model executes the forward pass using the updated weights. Because the architecture of the base model remains strictly unchanged, the per-token complexity remains $O(T \cdot L \cdot d^2)$.

Total Cost. Combining these components, the total cost for a single generation is:

$$\mathcal{C}_{\text{total}} = \underbrace{O(T_s \cdot L_{\text{hyp}} \cdot d^2_{\text{hyp}})}_{\text{Hypernet Pass}} + \underbrace{O(|\mathcal{L}_{\text{steer}}| \cdot d^2 \cdot r)}_{\text{Weight Update}} + \underbrace{O(T \cdot L \cdot d^2)}_{\text{Standard Decoding}}$$

As $T \to \infty$, the setup costs (first two terms) vanish and the ratio of our method's cost to the baseline approaches 1. In activation steering, however, steering vectors are typically computed $\forall$ token steps t, introducing a linear overhead term $O(T \cdot d)$ that does not amortize.

5. Numerical Experiments

5.1 Implementation Details

We primarily utilize **Gemma-2B** as the base model due to computational constraints, though we ablate our choice of base model by also running experiments on **Gemma-9B**. We implement the weight steering updates using a low-rank decomposition with a rank of $r = 64$. The hypernetwork is instantiated as a Transformer decoder with a depth of 8 layers. We steer a targeted subset of the model parameters: specifically 2 layers starting from layer 10. The model is optimized for 10 epochs using standard Cross-Entropy (CE) loss. As a comparative analysis, we benchmark on AxBench and against the HyperSteer demo implementation (AlpacaEval), as the documentation of the full version is unreleased.

5.2 AxBench Dataset Generation Protocol

We train on a dataset of 896 000 tokens (500 concepts with 14 examples of 128 tokens per concept) generated with AxBench's synthetic data-generation protocol. The dataset consists of concepts that we want a model to follow for steering. Each concept is a short description (spanning text, code, and math domains). AxBench obtains examples in two ways: (i) slightly changing an existing text so the concept appears (ii) continuing an unfinished text. Each example is labeled with a number called the steering factor, which controls the intensity of the steering (a factor of 0.0 corresponds to unsteered behavior). We use AlpacaEval to test generalization to new data.

5.3 Training Objective and Optimization

For HyperSteer-Weight only the hypernetwork and low-rank bases are trained. We train on next token prediction with cross-entropy loss. The base LLM is frozen while gradients flow through the global (trained) LoRAs into the hypernetwork. We optimize HyperSteer-Weight using AdamW. The learning rate is kept constant at the beginning of training and then decreased linearly to zero after a chosen step. The loss decreases promptly during training, and follows a striking pattern of phase changes discussed in Appendix B.

5.3.1 Distributed Training

Training runs simultaneously on multiple GPUs using DDP. Each rank (GPU) gets its own split from the dataset. After each training step, all ranks synchronize so that they agree on parameter updates and the model stays identical. Each rank runs enough steps so that, together, all ranks process the full dataset. The step count per rank is total steps/rank = $\left\lceil \frac{|\mathcal{D}|}{\text{batch_size} \times N} \right\rceil$, where $|\mathcal{D}|$ is the dataset size and N is the number of ranks.

5.4 Evaluation Metrics

We evaluate steering with AxBench's automated judge and by computing perplexity.

The concept score – an integer in $\{0, 1, 2\}$ – measures how strongly an output shows the target concept according to the judge. It is averaged across concepts and examples, with higher values indicating stronger concept relevance.

The **composite score**, the metric used in AxBench, measures the overall quality of the output under steering. It is computed per example using the harmonic mean of concept relevance, instruction relevance, and fluency, each scored between 0 and 2. If any component is 0, the entire score becomes 0, forcing the model to perform well on all aspects.

Perplexity is used as a fluency signal, computed on the steered outputs: $e^{\mathcal{L}_{CE}}$. Lower values indicate more natural language, while higher values indicate less fluent text.

5.5 AxBench

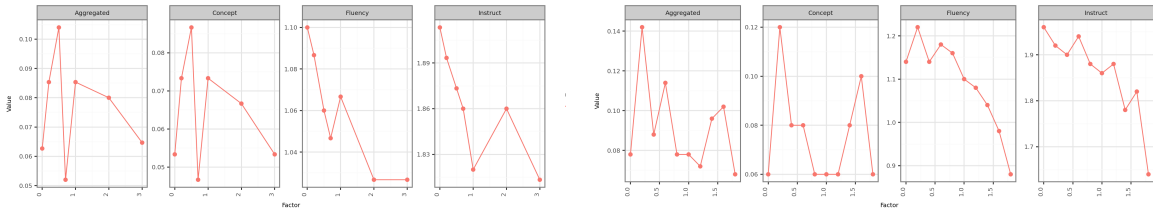
The AxBench score is the composite score over a set of 300 prompts steered towards concepts from the GemmaScope concept list (Lieberum et al. (2024)). We exceed baselines such as DiffMean and unsteered generations (see ablations below). While HyperSteer-Weight's performance is not state-of-the-art, the method is unique in that it is the only weight steering method (SAEs perform activation steering) that does not require training prior to generation (RePS and ReFT need to be trained per-concept).

Method	Score
Prompt	0.894
RePS	0.720
ReFT-r1	0.543
SAE (filtered)	0.508
HyperSteer-Weight	**0.414**
DiffMean	0.239
SAE	0.165
SAE-A	0.157
LAT	0.127
PCA	0.105
Probe	0.098

Table 1: AxBench.

5.6 Comparing with HyperSteer on Alpaca Eval

AlpacaEval is an LLM-based evaluation benchmark that evaluates the quality of model outputs using pairwise comparisons on a diverse set of instruction-following prompts. It approximates human preference judgments and is often used to evaluate generalization and overall response quality. We evaluate HyperSteer-Weight on AlpacaEval to assess generalization beyond AxBench concepts and compare its performance directly to HyperSteer under the same evaluation protocol, as HyperSteer only open sourced a demo showcasing performance on AlpacaEval (hence we do not compare to it on AxBench).



(a) HyperSteer-Weight (b) HyperSteer

Figure 2: Comparison of AlpacaEval Performance

Results are comparable, with HyperSteer-Weight matching HyperSteer. For both methods, we observe *oversteering*: factors < 1 outperform raw hypernetwork outputs (LoRAs/activations at factor $= 1$). This is further evidenced by the degradation in instruction following and fluency for factors > 1, which suggests the counterintuitive heuristic that factors < 1 should be selected, as oversteering is more likely than understeering.

5.7 Ablations

We conducted *ablation* – additional experiments with a change one hyperparameter at a time while keeping all others fixed, in order to observe how performance changes and identify which factors matter most. Specifically, we ablate: (1) the LoRA rank (higher rank corresponds to higher capacity); (2) the number of epochs; (3) the intervention layer, i.e., at which model layer steering starts to be applied to; (4) the number of layers modified; (5) the hypernetwork depth, which controls the complexity of the hypernetwork; (6) the base model size within the Gemma family.

We compare results in terms of perplexity, CE loss, improvement in aggregate score on AxBench, and improvement in score on AlpacaEval. Improvements are more informative than absolute scores in this context because the baseline (factor 0) has a positive score (due to some of the concepts in AxBench/AlpacaEval occurring in the prompt, for instance programming-related concepts appearing in coding prompts). We observe overfitting across runs (3), (4), and (6). This is evidenced by the lower CE Loss/Perplexity not contributing to better performance on either AxBench or AlpacaEval. All these runs either increase parameter count or the number/depth of steered layers, which facilitates overfitting. We report details concerning statistical significance in Appendix C.

Table 2: Ablation study results. We report Perplexity (PPL) and Cross-Entropy (CE) on the training set. For AxBench (train) and AlpacaEval (test), we show the Mean ± Std score, the improvement over the unsteered baseline (Δ), and the p-value.

			AxBench (Train)			AlpacaEval (Test)		
Ablation	PPL ↓	CE ↓	Score ↑	Δ ↑	p-val	Score ↑	Δ ↑	p-val
Default (HyperSteer-Weight)	2.77	1.083	0.4140 ± 0.0306	0.1007	0.0072	0.1040 ± 0.0109	0.0413	0.0010
(1) Rank $r = 4$	3.31	1.246	0.3627 ± 0.0293	0.0507	0.1067	0.0887 ± 0.0085	0.0000	0.5000
(2) Epochs $N = 3$	6.44	1.632	0.3700 ± 0.0294	0.0853	0.0128	0.0787 ± 0.0083	0.0060	0.3039
(3) Start Layer $l = 20$	1.75	0.752	0.3447 ± 0.0296	0.0520	0.0905	0.0867 ± 0.0091	0.0000	0.5000
(4) Modified Layers $k = 6$	2.07	0.839	0.3206 ± 0.0259	0.0494	0.0873	0.0486 ± 0.0048	0.0040	0.2730
(5) HNet Depth $d = 4$	2.12	0.841	0.3733 ± 0.0328	0.0773	0.0326	0.0633 ± 0.0078	0.0167	0.0404
(6) Base Model: Gemma-9B	1.23	0.135	0.3227 ± 0.0242	0.0207	0.2827	0.0820 ± 0.0080	0.0000	0.5000

6. Conclusion

Overall, HyperSteer-Weight is a meaningful step towards 0-shot fine-tuning, scores competitively on AxBench, and has advantages over other approaches such as RePS/ReFT, which require per-concept training, or SAEs, which operate in activation space and require considerable effort to filter the features down to an effective steering vector. We observe generalization to unseen prompts and domains on AlpacaEval.

A limitation of our method is that it's not state-of-the-art on AxBench. While our method has other distinct advantages, enhancing performance remains a crucial future direction. Furthermore, overfitting occurs across ablations, which could suggest HyperSteer also suffers from it to some extent. Finally, even an unsteered LLM will obtain a positive score on AxBench (prompts/concepts overlap) suggesting leakage, which we mitigate by extending our evaluation to other criteria such as perplexity and generation on AlpacaEval.

As future directions, regularization should be explored to enhance generalization in HyperSteer-Weight both to AlpacaEval and to generation on AxBench, which is distinct from training, which focuses on minimizing CE as a proxy rather than maximizing the AxBench score directly. Regularization could be introduced by adding weight decay to the Hypernetwork and/or by adding an auxiliary KL divergence loss with the unsteered model. Moreover, further experiments could be run on Gemma-9B, as regularizing might yield greater benefits in a larger model.

References

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, 2020. URL `https://papers.nips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html`.

Bowen Cao, Deng Cai, Zhisong Zhang, Yuexian Zou, and Wai Lam. On the worst prompt performance of large language models. In *Advances in Neural Information Processing Systems*, 2024. URL `https://proceedings.neurips.cc/paper_files/paper/2024/hash/7fa5a377b7ffabcce43cd00231bb3f9c-Abstract-Conference.html`.

Damai Dai, Li Dong, Yaru Hao, Zhifang Sui, Baobao Chang, and Furu Wei. Knowledge neurons in pretrained transformers. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 8493–8502, 2022. doi: 10.18653/v1/2022.acl-long.581. URL `https://aclanthology.org/2022.acl-long.581/`.

Constanza Fierro and Fabien Roger. Steering language models with weight arithmetic. *arXiv preprint arXiv:2511.05408*, 2025. doi: 10.48550/arXiv.2511.05408. URL `https://arxiv.org/abs/2511.05408`.

Antonio Andrea Gargiulo, Donato Crisostomi, Maria Sofia Bucarelli, Simone Scardapane, Fabrizio Silvestri, and Emanuele Rodolà. Task singular vectors: Reducing task interference in model merging. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18695–18705, 2025. URL `https://openaccess.thecvf.com/content/CVPR2025/html/Gargiulo_Task_Singular_Vectors_Reducing_Task_Interference_in_Model_Merging_CVPR_2025_paper.html`.

David Ha, Andrew M. Dai, and Quoc V. Le. Hypernetworks. In *International Conference on Learning Representations*, 2017. URL `https://openreview.net/forum?id=rkpACe1lx`.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *International Conference on Machine Learning*, pages 2790–2799, 2019. URL `https://proceedings.mlr.press/v97/houlsby19a.html`.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=nZeVKeeFYf9.

Robert Huben, Hoagy Cunningham, Logan Riggs Smith, Aidan Ewart, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=F76bwRSLeK.

Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=6t0Kwf8-jrj.

Shruti Joshi, Andrea Dittadi, Sébastien Lachapelle, and Dhanya Sridhar. Identifiable steering via sparse autoencoding of multi-concept shifts. *arXiv preprint arXiv:2502.12179*, 2025. doi: 10.48550/arXiv.2502.12179. URL https://arxiv.org/abs/2502.12179.

Feyza Duman Keles, Pruthuvi Maithri Wijewardena, and Chinmay Hegde. On the computational complexity of self-attention. In *Algorithmic Learning Theory (ALT)*, pages 597–619, 2023. URL https://proceedings.mlr.press/v201/duman-keles23a.html.

Dong-Ki Kim, Miao Liu, Matthew Riemer, Chuangchuang Sun, Marwa Abdulhai, Golnaz Habibi, Sebastian Lopez-Cot, Gerald Tesauro, and Jonathan P. How. A policy gradient algorithm for learning to learn in multiagent reinforcement learning, 2021. URL https://arxiv.org/abs/2011.00382.

Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics and the International Joint Conference on Natural Language Processing (ACL-IJCNLP)*, pages 4582–4597, 2021. doi: 10.18653/v1/2021.acl-long.353. URL https://aclanthology.org/2021.acl-long.353/.

Tom Lieberum, Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Nicolas Sonnerat, Vikrant Varma, Janos Kramar, Anca Dragan, Rohin Shah, and Neel Nanda. Gemma scope: Open sparse autoencoders everywhere all at once on gemma 2. In *Proceedings of the BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pages 278–300, 2024. doi: 10.18653/v1/2024.blackboxnlp-1.19. URL https://aclanthology.org/2024.blackboxnlp-1.19/.

Tianyang Lin, Yuxin Wang, Xiangyang Liu, and Xipeng Qiu. A survey of transformers. *AI Open*, 3:111–132, 2022. doi: 10.1016/j.aiopen.2022.10.001. URL https://www.sciencedirect.com/science/article/pii/S2666651022000146.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/6f1d43d5a82a37e89b0665b33bf3a182-Abstract-Conference.html.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html.

Jason Phang, Yi Mao, Pengcheng He, and Weizhu Chen. HyperTuning: Toward adapting large language models without back-propagation. In *International Conference on Machine Learning*, pages 27854–27875, 2023. URL https://proceedings.mlr.press/v202/phang23a.html.

Nina Rimsky, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Turner. Steering Llama 2 via contrastive activation addition. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 15504–15522, 2024. doi: 10.18653/v1/2024.acl-long.828. URL https://aclanthology.org/2024.acl-long.828/.

Jiuding Sun, Sidharth Baskaran, Zhengxuan Wu, Michael Sklar, Christopher Potts, and Atticus Geiger. Hypersteer: Activation steering at scale with hypernetworks. *arXiv preprint arXiv:2506.03292*, 2025. doi: 10.48550/arXiv.2506.03292. URL https://arxiv.org/abs/2506.03292.

Richard S. Sutton and Andrew G. Barto. *Reinforcement learning : an introduction*. Adaptive computation and machine learning series. The MIT Press, Cambridge, Massachusetts, second edition. edition, 2018. ISBN 9780262039246.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. URL https://papers.nips.cc/paper/7181-attention-is-all-you-need.

Tomer Volk, Eyal Ben-David, Ohad Amosy, Gal Chechik, and Roi Reichart. Example-based hypernetworks for multi-source adaptation to unseen domains. In *Findings of the Association for Computational Linguistics: EMNLP*, pages 9096–9113, 2023. doi: 10.18653/v1/2023.findings-emnlp.610. URL https://aclanthology.org/2023.findings-emnlp.610/.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=1PL1NIMMrw.

Yihan Wang, Si Si, Daliang Li, Michal Lukasik, Felix Yu, Cho-Jui Hsieh, Inderjit S. Dhillon, and Sanjiv Kumar. Two-stage LLM fine-tuning with less specialization and more generalization. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=pCEgna6Qco.

Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*, 2022a. URL https://openreview.net/forum?id=gEZrGCozdqR.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022b. URL https://proceedings.neurips.cc/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html.

Zhengxuan Wu, Aryaman Arora, Atticus Geiger, Zheng Wang, Jing Huang, Dan Jurafsky, Christopher D. Manning, and Christopher Potts. AxBench: Steering LLMs? Even simple baselines outperform sparse autoencoders. In *International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=K2CckZjNy0.

Biao Zhang, Zhongtao Liu, Colin Cherry, and Orhan Firat. When scaling meets LLM finetuning: The effect of data, model and finetuning method. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=5HCnKDeTws.

Appendix A. Steering Example

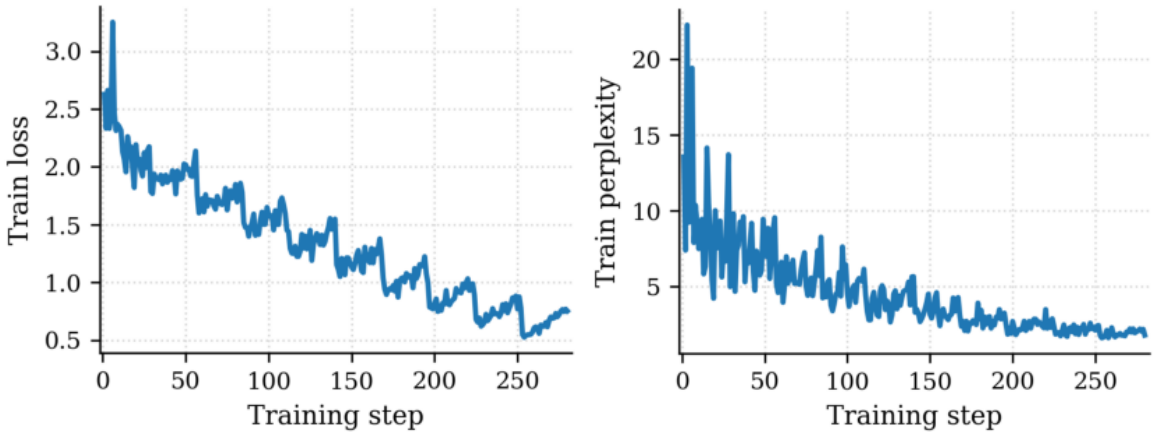
To illustrate the workflow, consider a steering scenario from the AxBench dataset.

- **Concept Tag (s):** `"simple vocabulary"`
- **Hypernetwork Action:** $H_\phi(\text{"simple vocabulary"}) \rightarrow \Delta\theta_{simple}$
- **User Input (x):** "Explain quantum mechanics."

The hypernetwork processes the concept tag once, updates the base model weights to favor simpler lexicon, and the model generates a simplified explanation for x without further intervention.

Appendix B. Phase changes during training

We observe a distinct pattern of phase changes occurring at regular intervals. This is in line with recent work Gargiulo et al. (2025) on learning theory across different concepts, where most of the loss decreases are explained by a few principal components in weight space being learned. We interpret the phase changes as being the point in training when such principal components are learned by the LoRA matrices and can hence be selected by the concept coefficients, which happens suddenly after a longer phase of learning the principal component.



(a) Cross-entropy loss

(b) Perplexity

Figure 3: Evolution of HyperSteer-Weight across training steps.

Appendix C. Statistical Significance

For each evaluation run, we compute the composite score for each steering factor by averaging the scores across all concepts. This gives an overall score per factor. For both the best factor and the baseline, we report the mean score and its standard error. Standard errors are computed from the variation across concepts/prompts and scaled using a fixed sample size of 300. We also report one sided p-values (with the hypothesis being no improvement from HyperSteer-Weight over the baseline) calculated with Z-scores from the normal approximation of the samples means given by the Central Limit Theorem.

Appendix D. Hyperparameters

We report hyperparameters for training and inference with HyperSteer-Weight for reproducibility.

Appendix E. Implementation Details for Weight Editing

The weight merging is implemented efficiently as a parallel branch (akin to LoRA), it's algebraically equivalent to adding a prompt-conditioned edit to model parameters θ *prior* to generation. This is done because editing the weights at train time takes much longer than implementing the forward pass as an extra parallel computation in PyTorch (as a wrapper). This is not necessary for inference at all, where the weight might actually be merged prior to generation without using a wrapper.

Table 3: Training hyperparameters for HyperSteer-Weight reproducibility.

Parameter	**Value**
Model Configuration	
Base Model / Hypernet Path	`google/gemma-2-2b-it`
Hypernet Layers	8
Low-rank Dimension (r)	64
Target Modules	`mlp.up_proj`, `mlp.down_proj`
Target Layers	10, 11
Initialize from Pretrained	False
Optimization	
Optimizer / LR	Adam / 1e-3
Learning Rate Min	0.0
LR Decay Start Step	600
Weight Decay	0.0
Training Epochs	10
Batch Size	16
Grad. Accumulation	1
Precision	`bfloat16`
Data & Training Logic	
Intervention Positions	All
Binarize Dataset	False
Train on Negative	False
Exclude BOS	True
Seed	42